

```
(gdb) b thread_entry
Breakpoint 1 at 0x804858c: file ify_thread_2.c, line 33.
(gdb) run

(gdb) info threads
Id      Target Id      Frame
4      Thread 0xb6feab70 (LWP 2507) "mythread" thread_entry
(arg=0x80487b7)
    at ify_thread_2.c:33
3      Thread 0xb77ebb70 (LWP 2506) "mythread" thread_entry
(arg=0x80487a9)
    at ify_thread_2.c:33
2      Thread 0xb77ecb70 (LWP 2505) "mythread" thread_entry
(arg=0x80487a0)
    at ify_thread_2.c:33
* 1      Thread 0xb7fed6c0 (LWP 2502) "mythread" (running)
(gdb)
```

In the above snippet, you see that there are four threads in total. (In my code, there are three calls to *pthread_create*, and one main thread.) GDB associated its own integer numbers 1-4 with the threads. The current thread is marked with a * and is in the running state. The 'arg' mentioned at the right is the argument given to the thread's starting function.

Now, you can switch your focus to some other threads by using the GDB command *thread threadnumber*.

```
(gdb) thread 2
[Switching to thread 2 (Thread 0xb77ecb70 (LWP 2505))]
#0  thread_entry (arg=0x80487a0) at ify_thread_2.c:33
33  retstr = strconvert((char*)arg);
(gdb) info threads
Id      Target Id      Frame
4      Thread 0xb6feab70 (LWP 2507) "mythread" thread_entry
(arg=0x80487b7)
    at ify_thread_2.c:33
3      Thread 0xb77ebb70 (LWP 2506) "mythread" thread_entry
(arg=0x80487a9)
    at ify_thread_2.c:33
* 2      Thread 0xb77ecb70 (LWP 2505) "mythread" thread_entry
(arg=0x80487a0)
    at ify_thread_2.c:33
1      Thread 0xb7fed6c0 (LWP 2502) "mythread" (running)
(gdb)
```

Notice that Thread 2 has become the current thread.

Let's assume you are printing some value in the current thread, and then continuing (&C) the execution of the thread. The current thread exited, and you can now switch focus to other threads, which are possibly waiting at breakpoints.

```
(gdb) printf "%s", arg
ABCDEFFG
(gdb) c&
```

```
Continuing.
Abcdeffg
(gdb) [Thread 0xb7fecb70 (LWP 2505) exited]
```

Now Thread 2 has exited, which can be seen from the *info threads* command below:

```
(gdb) info threads
Id      Target Id      Frame
4      Thread 0xb6feab70 (LWP 2507) "mythread" thread_entry
(arg=0x80487b7)
    at ify_thread_2.c:33
3      Thread 0xb77ebb70 (LWP 2506) "mythread" thread_entry
(arg=0x80487a9)
    at ify_thread_2.c:33
1      Thread 0xb7fed6c0 (LWP 2502) "mythread" (running)
```

```
The current thread <Thread ID 2> has terminated. See 'help thread'.
(gdb)
```

So, in this way, you can walk through a multi-threaded program using GDB. This is quite a good way to learn. I would suggest you download the GDB manual, and read the relevant section too. Also, some of the reference links suggested here are quite good for getting a better understanding of the topic. I hope you grasped the basics of writing and debugging multi-threaded applications. Now you can consider applying multi-threading to some of your existing or future applications. **END** 

References

- [1] Stevens, W. Richard and Rago, Stephen A. 'Advanced Programming in the Unix Environment,' 2nd Edition, Pearson Education
- [2] Butenhof, David R, 'Programming with POSIX® Threads,' Addison Wesley
- [3] Linux manual pages
- [4] <https://computing.lnl.gov/tutorials/pthreads/>
- [5] <http://red.ht/RLbrCA>
- [6] <http://www.yolinux.com/TUTORIALS/LinuxTutorialPosixThreads.html>

Acknowledgement

I would like to thank Tanmoy Bandyopadhyay and Vikas Nagpal for their help in reviewing this article.

By: Swati Mukhopadhyay

The author has more than 12 years of experience in academics and corporate training. Her interests are digital logic, computer architecture, computer networking, data structures, Linux and programming languages like C and C++. You can contact her at swati.mukerjee@gmail.com or <http://swatimukhopadhyay.wordpress.com/>